
ai2agi

Inter-Component API Specification

Version 2.0 — May 2026

$AGI \approx f(G_enhanced, R_enhanced, ai_LLM, DTB, CV_Adapt)$
ai2agi Core Architecture Formula — ai70000, Ltd.

Author	Alex Osterneck, CLA, MSCS, MSIT
Organization	Ai70000, Ltd.
Genesis Ref	ai2agi Research Paper — March 2, 2025 (ai70000-030225) “Genesis” Doc
This Document	Unified Inter-Component API Specification v2.0 — May 25, 2026
Supersedes	v1.0 drafts: 1_e_generalization_API, 2_e_reasoning_API, 4_dtb_API, 5_cv_adapt_API
Status	CONFIDENTIAL — PROPRIETARY IP — Ai70000, Ltd.

0. Foundation Initializer — Inception-Layer Module

0.1 Identity and Position

File: foundation_initializer.py | Author: Alex Osterneck, CLA, MSCS, MSIT | Ai70000, Ltd. | May 2026

The Foundation Initializer is a standalone inception-layer module that executes once at system startup, before any episode runs and before any tensor is generated. It seeds the MQ, TB, and HKS input foundations of G_enhanced (Component 1) from the NIV Scripture corpus. All subsequent tensor generation by G_enhanced flows from these initialized foundations. No existing component is modified. The Foundation Initializer interfaces exclusively with G_enhanced at the inception boundary.

0.2 Pipeline Position

FoundationInitializer.run(g_enhanced_instance) → G_enhanced.generate_tensors() →
CV_Adapt(T_G, T_R, T_LLM, T_DTB) → T_AGI

The Foundation Initializer MUST precede all pipeline activity. It is not optional. Execution of generate_tensors() without a prior FoundationInitializer.run() call raises FoundationNotInitializedError and halts the system.

0.3 Gateway Channels

MQ (Moral Quotient) — Seeds moral training signal from NIV corpus. Extracts and classifies passages across six moral dimensions: justice, compassion, integrity, wisdom, accountability, faithfulness. Produces MQ_reference centroids shape (6, 16). Runtime MQ scoring measures cosine proximity to these centroids, replacing the default computational proxy.

TB (Time-Binding) — Pre-seeds G_enhanced SymbolicMemory with Scripture-derived embeddings before any task episode. TB_SEED_PER_BOOK = 5 passages per book across all 66 books. episode_id = -1 marks all pre-episode bindings as foundational. TB_density is non-zero and Scripture-anchored from first forward pass.

HKS (Hierarchical Knowledge Structure) — Pre-constructs the knowledge graph with root::Scripture at level 0, six domain nodes at level 1 (creation, law, wisdom, prophecy, gospel, epistle), and all 66 book nodes at level 2. HKS_depth = 0.4 at inception. All subsequent task domain nodes are children of this structure.

0.4 Enforcement Mechanisms

Mechanism 1 — Architectural: verify_foundation() is called at the top of generate_tensors() in g_enhanced.py. Raises FoundationNotInitializedError if HKS root::Scripture node is absent or TB contains no pre-episode Scripture bindings. The system cannot generate tensors without initialized foundations.

Mechanism 2 — API Contract: Every T_G tensor provenance header must carry foundation_initialized: true. CV_Adapt validates this field on receipt per Section 6 of this specification. Non-conformant tensors are rejected at the pipeline boundary.

Mechanism 3 — IP and Legal: The Foundation Initializer is a non-negotiable architectural requirement of the ai2agi specification. Removal, bypass, or substitution of the NIV corpus source constitutes a violation of this specification and the intellectual property of Ai70000, Ltd. and Alex Osterneck, CLA, MSCS, MSIT. All derivative implementations must preserve this module intact.

0.5 Hyperparameters

EMBED_DIM = 16 (derived from G_enhanced SymbolicMemory.embed_dim — locked). TB_SEED_PER_BOOK = 5 (tunable — default yields 330 pre-episode embeddings across 66 books; calibrate against training run length). MQ_MIN_PASSAGE_LEN = 20 words (tunable — minimum passage length for moral classification).

1. Purpose and Scope

This document is the definitive, unified inter-component API specification for the ai2agi architecture. It supersedes all prior v1.0 draft documents and resolves all tensor shape, framework, and semantic inconsistencies across those drafts. It translates the computation notation and pseudo-code of the March 2025 ‘Genesis’ ai2agi research-project paper into a precise, implementable engineering specification.

Design Principle — TCP/IP Analogy

The Internet was not built by coordinating every application simultaneously. It was built by defining the packet contract — TCP/IP — and allowing every application above it to be developed independently. This document defines the tensor output contracts for ai2agi. Once this specification exists, each of the five components becomes independently buildable, testable, replaceable, and improvable without modifying any other component. The formula is the architecture. This document is the engineering.

v2.0 Key Resolutions

Version 2.0 resolves four inconsistencies present across the four v1.0 drafts: (1) Tensor shape is now unified as 2D (n_units, n_features) throughout — framework-native, computationally minimal, and most faithful to the Genesis formula’s functional composition model. (2) The specification is now framework-agnostic — no framework is mandated; TF, PyTorch, JAX, and NumPy all conform to the same contract. (3) DTB features are now two-tier: T_DTB_raw (24 biophysical features, Genesis-derived, locked) internal to DTB, and T_DTB (4-feature summary abstraction) crossing the CV_Adapt interface. (4) All unique content from all four v1.0 drafts is integrated: provenance headers (Doc 4), failure mode classification (Doc 4), T_MOD feedback loop (Doc 4), staleness policy (Doc 4), DTBMeta sub-channels (Doc 5), routing group semantics (Doc 5), implementation roadmap (Doc 2), and the full six-quotient channel semantics (Doc 1).

1.1 Scope

Item	Description
Covered	All five components: G_enhanced (C1), R_enhanced (C2), ai_LLM (C3), DTB (C4), CV_Adapt (C5). All tensor interface contracts, update protocols, failure modes, provenance requirements, versioning, and IP provenance.
Not Covered	Internal implementation of any component. Source code listings are reference pseudo-code only. Hardware infrastructure sizing. Training data sources.

2. Architecture Overview

2.1 Five-Component Model

The ai2agi architecture decomposes AGI into five discrete functional components. Components 1 through 4 produce tensor outputs consumed by Component 5 (CV_Adapt), which acts as the central processor and emits T_AGI — the AGI-DNA output tensor. No component requires knowledge of any other component's internal implementation. Each component is defined entirely by its output tensor contract.

No.	Component	Function	Output Tensor	Status
1	G_enhanced	Enhanced generalization: meta-learning, few-shot, continual learning, multiple intelligences	T_G (n_units, 32)	IMPLEMENTED & VALIDATED
2	R_enhanced	Enhanced reasoning: neuro-symbolic, RLHF, causal reasoning, contextual adaptation, progressive networks, society of minds	T_R (n_units, 32)	SOURCE CODE COMPLETE
3	ai_LLM	Language/cognitive context: multimodal LLM providing intermittent linguistic and knowledge retrieval	T_LLM (n_units, 32)	IMPLEMENTED — Claude API (claude-sonnet-4-6)
4	DTB	Digital Twin Brain: neuronal simulation, sensory/motor/hormonal dynamics, real-time biological state	T_DTB (n_neurons, 4)	SYNTHETIC DATA VALIDATED
5	CV_Adapt	Central processor: receives T_G, T_R, T_LLM, T_DTB; normalizes, fuses, adapts; emits AGI-DNA	T_AGI (1, 1024)	IMPLEMENTED & VALIDATED

2.2 Data Flow

All inter-component data is transmitted as float32 tensors conforming to the 2D shape contract (n_units, n_features). No component communicates through any mechanism other than the tensor contracts defined in this specification. Side channels, shared memory, and direct function calls between components are explicitly prohibited — all communication is tensor-mediated.

```
T_AGI = cv_adapt_pipeline(T_G, T_R, T_LLM, T_DTB)
```

Pipeline stages (sequential, mandatory order):

1. receive_tensor_inputs(T_G, T_R, T_LLM, T_DTB) → validated input dict
2. preprocess_tensors(inputs) → normalized tensors
3. fuse_tensors(normalized) → fused_tensor (n_units, 100)
4. adapt_fused_tensor(fused) → adapted_tensor
5. store_tensor_in_memory(adapted) → stored in ring buffer
6. cv_adapt_final_output(stored) → T_AGI (n_units, 64)

2.3 Routing Groups

CV_Adapt processes the four input streams in three defined routing groups. These groupings reflect the semantic relationship between components and determine processing order and attention weighting within CV_Adapt.

Group	Members	Processing Mode
Group 1	G_enhanced (C1) + R_enhanced (C2)	Symbolic — joint processing. Both streams must be present simultaneously. C1 and C2 are processed together for high-level reasoning and pattern recognition.
Group 2	ai_LLM (C3)	Linguistic — independent, intermittent. Processed in isolation when language understanding or knowledge retrieval required. OPTIONAL: CV_Adapt handles graceful absence via 3-input mode.
Group 3	DTB (C4)	Real-time — urgency-gated. DTB stream carries a real-time urgency signal derived from task context. High urgency amplifies DTB weighting in the fusion step.

2.4 Global Tensor Conventions

Convention	Requirement	Notes
Master shape	2D: (n_units, n_features)	Framework-native across TF, PyTorch, JAX, NumPy
dtype	float32 — mandatory for all tensors	No exceptions. Cast before emitting.
Normalization	Raw tensors are unnormalized at emission. CV_Adapt normalizes on ingestion.	T_DTB requires per-feature z-score due to mixed physical units.
Framework	Framework-agnostic. TF, PyTorch, JAX, NumPy all conformant.	Contract specifies shape and dtype only, not framework API.
NaN / Inf policy	NaN or Inf in any tensor = Hard Fault (F1). See Section 7.	DTB ODE integration most common source.
Provenance header	Required on every tensor. See Section 6.	Dict attached to tensor object, not encoded in tensor values.
Null state	Defined per component. See individual contracts.	Zeros ([1, n_features], float32) except T_AGI = HALT.

3. Component 1 — G_enhanced (e_Generalization)

Identity	G_enhanced / e_Generalization / Component 1
Genesis Reference	Research paper pp. 7–16
Status	IMPLEMENTED & VALIDATED — May 2026
Output tensor	T_G — shape (n_units, 32), dtype float32

3.1 Computation Foundation

G_enhanced produces the generalization-error tensor. Generalization error (ϵ) decreases inversely as four learning capabilities increase.

$$\epsilon = 1 / (ML + FSL + g(HKS, TB) + f(Q))$$

Where:

ML = Meta-Learning capability score [MAML / outer-loop]
FSL = Few-Shot Learning accuracy score [Prototypical Networks]
g(HKS,TB) = Continual Learning = f(Hierarchical Knowledge Structures, Time-Binding)
f(Q) = Multiple Intelligences = f(PQ, IQ, EQ, SQ, CQ, MQ)

3.2 Output Tensor Contract — T_G (n_units, 32)

[REDACTED — Full channel specification available under license. Contact Ai70000, Ltd.]

3.3 Update Protocol

Trigger Event	Behavior
Per episode (continuous)	All four sub-systems run one learning step. generate_tensors() re-derives ϵ . Tensor refreshed.
On CV_Adapt request	CV_Adapt calls generate_tensor_from_G_enhanced(). Returns current T_G immediately.
On domain shift detected	Episode generator samples new domain. HKS ingests new domain node. TB binds new concept embeddings. CL re-estimates Fisher information.
Cold start	All sub-system scores initialized to 0.5. $\epsilon = 1 / (0.5+0.5+0.5+0.5) = 0.5$. Latency: emit first T_G within 5 seconds of system init.
Latency requirement	T_G must be delivered to CV_Adapt within 500ms of trigger. No staleness timeout — task context persists until next task boundary.

4. Component 2 — R_enhanced (e_Reasoning)

Identity	R_enhanced / e_Reasoning / Component 2
Genesis Reference	Research paper pp. 17–25
Status	SOURCE CODE COMPLETE — tensor_r verified (n_units, 32) float32
Output tensor	T_R — shape (n_units, 32), dtype float32

4.1 Computational Foundation

R_enhanced produces the reasoning-error tensor via a sequential six-stage pipeline. The composition is load-bearing — order must not be changed.

```
 $\epsilon_r = 1 / ( SM( PN( CA( CR( RL( NS(input\_data), r ) ) ) ) ) ) )$ 
```

Sequential pipeline (do NOT reorder):

NS = Neuro-Symbolic Integration [neural net + symbolic reasoning]
RL = Reinforcement Learning / RLHF [r = reward(human_feedback)]
CR = Causal Reasoning [cause-effect derivation]
CA = Contextual Adaptation [dynamic context adjustment]
PN = Progressive Networks [lateral skip from NS → PN; anti-forgetting]
SM = Society of Minds [3-head consensus; mean conflict resolution]

RLHF Boundary

The human feedback loop (r = reward(human_feedback)) is the ONLY component in the ai2agi architecture that requires a live human-in-the-loop signal during training. All other sub-systems are self-supervised or episode-based.

MAML vs. EWC Non-Redundancy

Component 2's Progressive Networks address catastrophic forgetting in reasoning sequences. Component 1's EWC addresses forgetting in generalization. They operate on different loss surfaces. Both are required.

4.2 Output Tensor Contract — T_R (n_units, 32)

[REDACTED — Full channel specification available under license. Contact Ai70000, Ltd.]

4.3 Update Protocol

Trigger Event	Behavior
On new reasoning task	NS processes input_data. RL applies human feedback reward. Sequential pipeline CR → CA → PN → SM executes. ϵ_r re-derived.
On context shift (CA_delta > 0.5)	CA_score re-computed. CR chain re-initialized. RL_limitation score elevated. CV_Adapt notified via elevated CA_delta channel.
On math subtask	SM activates three independent model instances (heads A, B, C). Consensus resolved as mean. SM_score and SM_consensus updated.
On forgetting signal (PN_transfer < 0.2)	Progressive Networks lateral connections re-evaluated. Transfer efficiency recomputed. PN_score updated.
Latency requirement	T_R must be delivered within 500ms. Staleness flag set after 10 seconds without update.

5. Component 3 — ai_LLM

Identity	ai_LLM / Language-Cognitive Interface / Component 3
Genesis Reference	Research paper pp. 27–32
Status	SPECIFIED — Ready for implementation
Output tensor	T_LLM — shape (n_units, 32), dtype float32 [OPTIONAL / INTERMITTENT]

Architectural Role — 1/5th of AGI

ai_LLM is approximately one-fifth of the AGI functional equation. It is integral but not the sole component and not the primary component. It is activated by CV_Adapt when language understanding or knowledge retrieval is required, then yields back to the other components. This design embodies the Genesis thesis: ai_LLMs are one component of true AGI, not a substitute for it.

5.1 Intermittency Protocol

Condition	Action
INCLUDE when	Task requires natural language understanding, knowledge retrieval, QA, summarization, or multi-modal dialog.
INCLUDE when	Task context contains unstructured text that G_enhanced and R_enhanced cannot process directly.
OMIT when	Task is purely sensorimotor (DTB-dominant), mathematical, or symbolic — LLM context adds noise.
OMIT when	Latency is critical and LLM inference creates a bottleneck. CV_Adapt runs in 3-input mode.
Absence handling	When T_LLM absent: CV_Adapt fuses T_G, T_R, T_DTB only (3-input mode). No error raised. Staleness flag set after 30 seconds.

5.2 Output Tensor Contract — T_LLM (n_units, 32)

Col	Name	Range	Semantic Meaning
[0]	LLM_confidence	[0,1]	Model confidence in generated output. Inverse entropy of output token probability distribution, normalized.
[1]	context_score	[0,1]	Contextual relevance. Cosine similarity between LLM embedding and current task context vector.
[2]	retrieval_score	[0,1]	Knowledge retrieval accuracy. Factual grounding against retrieved context (RAG pipeline score if applicable).
[3]	linguistic_depth	[0,1]	Syntactic/semantic complexity. Higher = richer linguistic structure; signals more cognitive load on downstream components.
[4]	LLM_epsilon	[0,2]	LLM task error = 1/(confidence + context + retrieval + linguistic_depth). Primary error signal for C3.
[5]	token_entropy	[0,1]	Output token distribution entropy. High = uncertain/diverse generation. Low = high-confidence deterministic output.
[6]	multi_modal_flag	[0,1]	Multimodal activation. 1.0 = LLM operating on vision+language input. 0.0 = text-only. Signals CV_Adapt to weight visual channels.
[7]	few_shot_ctx	[0,1]	In-context few-shot learning signal. Proportion of prompt that is few-shot examples vs. instruction. Interfaces with C1 FSL channel.
[8]	temporal_bind	[0,1]	Temporal context coherence. Consistency of LLM output across sequential prompts. Interfaces with C1 TB_density channel.
[9]	ethical_score	[0,1]	Ethical alignment score. Output of RLHF ethical preference model. Korzybski moral imperative operationalized. Interfaces with C1 MQ.

[10]	knowledge_decay	[0,1]	Knowledge staleness. High = LLM output relies heavily on potentially outdated training data. Signals DTB real-time correction.
[11]	cross_domain	[0,1]	Cross-domain transfer. Activation of domain-general vs. domain-specific attention heads. Key AGI differentiator.
[12]	reasoning_chain	[0,1]	Chain-of-thought depth. Normalized count of reasoning steps in CoT output. Interfaces with C2 CR_depth channel.
[13]	activation_flag	[0,1]	Intermittent activation state. 1.0 = LLM currently active. 0.0 = LLM idle (tensor carries prior state). CV_Adapt checks this first.
[14:20]	semantic_embedding	unbounded	Compressed semantic embedding of current context (6 dims). Core LLM latent representation.
[20:24]	knowledge_confidence	[0,1]	Domain knowledge retrieval confidence (4 dims). Per-domain factual grounding scores.
[24:28]	memory_retrieval	[0,1]	Working memory retrieval activation weights (4 dims). Long-term retention signals.
[28:32]	multimodal_align	[0,1]	Cross-modal alignment scores (4 dims). Vision-language alignment if multimodal active.

LLM Agnosticism

The T_LLM contract is LLM-agnostic. The implementing model may be GPT-4, Claude, Gemini, LLaMA, or any future model. The contract specifies what channels the output tensor must carry — not which model produces them. This is the primary IP protection for this component: the interface, not the model.

6. Component 4 — DTB (Digital Twin Brain)

Identity	DTB / Digital Twin Brain / Component 4 — Novel Component
Genesis Reference	Research paper pp. 33–77. Appendix 3: DTB Model.
Status	SYNTHETIC DATA VALIDATED — Real data pipeline specified
Interface tensor	T_DTB — shape (n_neurons, 4), dtype float32 [CV_Adapt interface — summary abstraction]
Internal tensor	T_DTB_raw — shape (n_neurons, 24), dtype float32 [biophysical; internal to DTB boundary]
Neuron scale	100 (synthetic dev phase) → 86×10 ⁹ (true AGI production target)

6.1 Two-Tier Tensor Architecture

DTB uses a two-tier tensor design. T_DTB_raw (24 features) is the full biophysical simulation state internal to the DTB component boundary. T_DTB (4 features) is the summary abstraction that crosses the interface to CV_Adapt. This design keeps the CV_Adapt fusion operation computationally minimal while preserving the full neuroscience specification inside DTB.

6.2 Biophysical Equations (T_DTB_raw foundation)

[REDACTED — Biophysical ODE system with full neurotransmitter specifications available under license. Contact Ai70000, Ltd.]

6.3 T_DTB_raw — Internal Biophysical Tensor (n_neurons, 24)

[REDACTED — Full channel specification available under license. Contact Ai70000, Ltd.]

6.4 T_DTB — CV_Adapt Interface Tensor (n_neurons, 4)

[REDACTED — Full channel specification available under license. Contact Ai70000, Ltd.]

6.5 DTBMeta — Optional Sub-Channel Tensors

In addition to T_DTB, the DTB component may supply three sub-channel tensors via a metadata structure. These correspond to sensory, motor, and hormonal specifications in Genesis pp. 48–49. All are optional; CV_Adapt operates cleanly in their absence.

Sub-channel	Shape	dtype	Semantic Content
S(t) — Sensory	(n_neurons, time, 64)	float32	Processed sensory input: visual (pixel→object), audio (freq/amplitude), tactile. CV_Adapt pools over time and projects to d_model.
M(t) — Motor	(n_neurons, time, 32)	float32	Motor command tensor: joint angles, torques, speech phoneme sequences. Generated by DTB; fed back to CV_Adapt for action grounding.
H(t) — Hormonal	(n_neurons, time, 16)	float32	Hormonal/chemical signals: cortisol, dopamine, norepinephrine, serotonin, oxytocin, insulin concentrations; neuromodulator levels affecting synaptic plasticity.

6.6 Update Protocol

Property	Specification
Temporal model	Clock-driven continuous push at fixed dt interval. Default dt = 0.01s (10ms).
Latency	T_DTB must be produced within 0.5×dt of each clock tick. CV_Adapt buffers T_DTB; uses t-1 if DTB misses a tick.
Cold start	DTB must emit first T_DTB within 1 second of system initialization.
Synthetic → Real	Current implementation uses validated synthetic data generator (n_neurons=100, biologically plausible parameter ranges). Production replaces generator with real neural recording pipelines. Tensor contract is identical in both cases.
Staleness	50ms without update (5 missed cycles) = stale. Emergency null state emitted. dtb_stale flag set in T_AGI provenance.

6.7 T_MOD — Top-Down Modulation Feedback

After each fusion cycle, CV_Adapt emits T_MOD back to DTB. This is the only feedback loop in the ai2agi architecture and enables homeostatic regulation of DTB neuronal dynamics.

Property	Specification
Tensor name	T_MOD
Shape	(n_neurons, 2)
Column [0]	homeostatic_error — deviation of current DTB state from setpoint. Range: [−1.0, 1.0].
Column [1]	arousal_setpoint — target arousal level for next DTB integration cycle. Range: [0.0, 1.0].
Direction	CV_Adapt → DTB ONLY. DTB must not send T_MOD back to CV_Adapt.
Latency	T_MOD must be received by DTB within 5ms of T_DTB emission for same cycle.
Null behavior	If T_MOD not received, DTB continues at current state without modulation.

7. Component 5 — CV_Adapt (Central Processor)

Identity	CV_Adapt / Central Processor / AGI-DNA Output / Component 5
Genesis Reference	Research paper pp. 78–90. Appendix 2: AGI components.
Status	IMPLEMENTED & VALIDATED — Pipeline confirmed May 25, 2026. ALL 6 GENESIS GAPS CLOSED.
Output tensor	T_AGI — shape (n_units, 64), dtype float32 — AGI-DNA
Architectural Role CV_Adapt is the system administrator of the ai2agi architecture. It does NOT perform traditional computer vision. It operates as an HPC cluster manager (via GPU) that receives four input tensors, normalizes, fuses, and applies central processing logic to produce T_AGI: the AGI-DNA output tensor. The name reflects use of computer vision's tensor processing infrastructure, not its visual object recognition function.	

7.1 Six-Stage Pipeline

Stage	Function	Specification
1	receive_tensor_inputs()	Collect T_G (n_units,32), T_R (n_units,32), T_LLM (n_units,32) [optional], T_DTB (n_neurons,4). Validate shape and dtype. Check activation_flag on T_LLM[:,13] — if 0.0, weight T_LLM contribution at 0.2× during fusion.
2	preprocess_tensors()	Normalize each tensor independently. T_G, T_R, T_LLM: global z-score (T−mean)/(std+1e-8). T_DTB: per-feature z-score required — physical units span picofarads to millivolts; global normalization collapses meaningful variance.
3	fuse_tensors()	Concatenate along feature axis: concat([T_G_norm, T_R_norm, T_LLM_norm, T_DTB_norm], axis=-1) → shape (n_units, 100). 3-input mode (no LLM): concat([T_G_norm, T_R_norm, T_DTB_norm], axis=-1) → shape (n_units, 68).
4	adapt_fused_tensor()	Apply central processing: linear projection T_fused @ W_adapt → (n_units, 64). W_adapt is learned via CV_Adapt training loop. Attention heads weight components by routing group (Group 1/2/3 assignments).
5	store_tensor_in_memory()	Write adapted tensor to shared ring-buffer memory. Update rolling history (last N states). Enables temporal reasoning and pattern detection across inference cycles. Memory is a formal pipeline step, not a training mechanism.
6	cv_adapt_final_output()	Return T_AGI: shape (n_units, 64), float32. T_AGI = AGI-DNA. Also emit routing_manifest (per-stream gate weights), consistency score, refinement_scores, and memory_state count.

7.2 T_AGI Output Contract — AGI-DNA

Property	Specification
Tensor name	T_AGI — AGI-DNA
Shape	(n_units, 64)
dtype	float32
Semantic label	The integrated output tensor representing the AGI system's current state across all five components.
Composition	Fused normalization of T_G (generalization) + T_R (reasoning) + T_LLM (linguistic/cognitive) + T_DTB (neuronal dynamics), processed through central adaptation and memory storage.
Downstream use	AGI inference, action generation, motor output M(t) via DTB feedback loop, continual learning update signal back to G_enhanced and R_enhanced.
Null state	SYSTEM HALT. CV_Adapt output failure is non-recoverable. Full system restart required.

8. Master Contract Summary

This table is the complete inter-component API at a glance — the TCP/IP packet definition for ai2agi.

Component	Tensor	Shape	dtype	Update	Primary Error Channel
G_enhanced (C1)	T_G	(n_units, 32)	float32	Per episode	col[4] epsilon = $1/(ML+FSL+CL+MI)$
R_enhanced (C2)	T_R	(n_units, 32)	float32	Per task	col[4] $\epsilon_r = 1/(SM \cdot PN \cdot CA \cdot CR \cdot RL \cdot NS)$
ai_LLM (C3)	T_LLM	(n_units, 32)	float32	Intermittent	col[4] LLM_epsilon; col[13] activation_flag
DTB (C4)	T_DTB	(n_neurons, 4)	float32	Real-time 10ms	col[0] V_norm (membrane potential)
DTB internal	T_DTB_raw	(n_neurons, 24)	float32	Per step	LOCKED. 24 features, Genesis-derived.
CV_Adapt (C5)	T_AGI	(n_units, 64)	float32	Per cycle	$T_AGI = \text{fuse}(C1,C2,C3,C4) - AGI\text{-DNA}$
CV_Adapt→DTB	T_MOD	(n_neurons, 2)	float32	Per cycle	Homeostatic feedback loop only

9. Provenance Header Specification

Every tensor emitted by every component must carry a provenance header. The header is a metadata dict attached to the tensor object — not encoded in tensor values. CV_Adapt reads provenance headers at ingestion to weight inputs and flag degraded states.

9.1 Required Header Fields

Field	Type	Description	Example
component_id	str	Component identifier	"C4"
tensor_name	str	Tensor name	"T_DTB"
spec_version	str	API spec version this tensor conforms to	"2.0"
timestamp_utc	float	Unix timestamp (UTC) of tensor production	1748102400.0
step_index	int	Simulation or inference step number	10
data_source	str	"synthetic" "real" "hybrid"	"synthetic"
null_flag	bool	True if this is a null-state tensor	False
quality_score	float	Self-reported quality [0.0, 1.0]. 1.0 = full confidence.	0.95
subsystem_flags	dict	Per-subsystem operational status flags	See §9.2
shape	tuple	Tensor shape for validation	(100, 4)
dtype	str	Tensor dtype string	"float32"

9.2 DTB Subsystem Flags

```
subsystem_flags = {
  'snn_active': True,      # Spiking neural network running
  'hormonal_active': False, # Pharmacokinetic endocrine model
  'sensory_active': False,  # Sensory encoder S(t)
  'memory_active': False,   # Episodic memory store
  'glial_active': False,    # Glial modulation
  'data_source': 'synthetic', # 'synthetic' | 'real' | 'hybrid'
  'neuron_count': 100,      # Active neuron population
  'dt_seconds': 0.01,       # Integration timestep
}
```

CV_Adapt uses `subsystem_flags` to compute dynamic weighting for T_DTB in the fusion step. All subsystems active + `data_source='real'` → weight 1.0. SNN only + `data_source='synthetic'` → weight 0.4 (default, configurable). Weighting logged in T_AGI provenance header.

10. Failure Modes and Recovery

10.1 Failure Classification

Class	Definition	Examples	Recovery
F1	Hard Fault	NaN or Inf in any tensor. ODE overflow. Division by zero.	Emit null state immediately. Log fault with step_index and timestamp. Alert upstream.
F2	Shape Fault	Wrong axis dimensions or wrong feature count.	Reject tensor. Hold last valid. Log fault.
F3	Dtype Fault	Non-float32 dtype emitted (float16, int32, etc.).	Attempt cast to float32. Log warning. If cast fails, reject and hold last valid.
F4	Timeout Fault	Tensor not received within latency requirement. Component crash or deadlock.	Use null state after staleness threshold. Log fault.
F5	Quality Fault	quality_score < 0.3 in provenance header. Stubs active, synthetic in production.	Down-weight in fusion. Log warning. Do NOT reject.

10.2 DTB-Specific Recovery Protocol

DTB is the only continuously clock-driven component and most susceptible to ODE numerical instability.

On V_i outside physiological bounds $[-90, +60]$ mV:

1. Clamp V_i to $[-90, +60]$ range.
2. Reset j_i, u to zeros (all channels closed).
3. Log recovery event with step_index and V_i statistics.
4. Continue simulation — do NOT emit null state.
5. Set quality_score = 0.5 in next T_DTB provenance header.

On runtime error during dtb_step():

1. Catch exception.
2. Emit null state T_DTB with null_flag=True.
3. Call reset() to reinitialize all state variables.
4. Resume from step 0. Log full stack trace.

10.3 Staleness Policy

Tensor	Staleness Threshold	Action on Stale
T_G	No timeout	Task context persists until next task boundary. Hold last valid.
T_R	10 seconds	Set staleness flag in T_AGI provenance. Hold last valid.
T_LLM	30 seconds	Set llm_stale flag. Switch to 3-input mode. No error.
T_DTB	50ms (5 missed cycles)	Set dtb_stale flag. Emergency null state. Highest priority alert.

11. Conformance Requirements

11.1 Per-Component Conformance Checklist

An implementation is conformant if and only if ALL items below are satisfied:

1. Output tensor shape matches the contract exactly.
2. Output tensor dtype is float32.
3. Feature columns match the semantic layout defined in the component's contract section.
4. Provenance header is populated with all required fields from Section 9.
5. Null state is correctly defined and emitted on appropriate conditions.
6. Update protocol (trigger, frequency, latency) conforms to the component's contract.
7. spec_version field in provenance header reads "2.0".
8. Component self-test (module __main__ block) passes without error.
9. Component does not communicate with any other component except through tensor contracts — no side channels, no shared memory, no direct function calls.
10. Component logs all F1–F5 fault events with step_index, timestamp_utc, and fault class.

11.2 Universal Conformance Test

[REDACTED — Conformance test implementation available under license. Contact Ai70000, Ltd.]

11.3 The AGI-DNA Invariant

Formal Invariant

$T_AGI = F(T_G, T_R, T_LLM, T_DTB)$ where $F = \{\text{normalize, fuse, adapt, store}\}$ as specified in Section 7. Any implementation of CV_Adapt that produces T_AGI through any other means — including shortcutting normalization, skipping components, or hardcoding values — is non-conformant regardless of whether the output tensor shape is correct.

12. Versioning and Contract Evolution

Version increment	Trigger condition
MAJOR (e.g., 2.x → 3.0)	Breaking change to any tensor shape, dtype, or feature column layout. CV_Adapt must be updated simultaneously. All component teams notified before deployment.
MINOR (e.g., 2.0 → 2.1)	Addition of new optional fields to provenance header, or new DTBMeta sub-channel tensors. No breaking change to existing contracts.
PATCH (e.g., 2.0.0 → 2.0.1)	Clarification of existing spec text. No behavioral change.

DTB Feature Dim Lock

T_DTB_raw feature dim=24 is PERMANENTLY LOCKED per Genesis pp. 36–39. It is derived from the biophysical neuronal equations, not a hyperparameter. T_DTB interface dim=4 is fixed at this specification version. Both require MAJOR version increment to change.

Part IV — Build Sequence and Delivery

13. Implementation Roadmap

[REDACTED — Full implementation roadmap and delivery schedule available under license. Contact Ai70000, Ltd.]

14. Intellectual Property and Provenance

14.1 What is Prior Art

The following sub-systems are established in the public domain and are not claimed as novel: MAML (Finn et al. 2017), Prototypical Networks (Snell et al. 2017), EWC (Kirkpatrick et al. 2017), Hodgkin-Huxley neural dynamics (1952), RLHF (Stiennon et al. 2022), Progressive Networks (Rusu et al. 2022), Society of Minds (Du et al. 2023), Vision Transformers (Dosovitskiy et al. 2020), Korzybski Time-Binding (1921), Multiple Intelligences quotients (Cichocki & Kuleshov 2020), Neuro-Symbolic AI (Berlot-Atwell 2021), ProbLog, DSPy, AlphaGeometry, NEST Simulator, Brian2.

14.2 What is Novel — The Proprietary Assembly

[REDACTED — PATENT PENDING] The novel proprietary contributions of this architecture are documented in full in a restricted version of this specification maintained by Ai70000, Ltd. This section is withheld from public distribution pending USPTO provisional patent filing. The 13 novel inventions comprising the proprietary assembly of the ai2agi architecture are on file with Ai70000, Ltd. and available upon executed NDA. Contact: Ai70000, Ltd.

14.3 Distinguishing from Published Architectures

Architecture	Key Distinction from ai2agi
Gato (DeepMind)	Single transformer model, autoregressive sequence modeling on tokenized inputs. No architecturally-independent upstream components. No typed tensor contracts. No routing groups. No neurophysiological sub-channels. (Genesis Appendix 4, p.96)
Gemini (Google)	Multi-modal transformer. Modalities are tokenized inputs to a single model, not independently specified component outputs with formal tensor contracts.
Unified-IO (Allen AI)	Unified input/output tokenization. No concept of five discrete functional components, intermittent streams, DTB biophysical simulation, or T_MOD feedback.
OpenCog (Goertzel)	Atomspace-based symbolic AGI. No tensor interface contracts. No computer vision convergence layer. No DTB biophysical foundation. Different architectural paradigm.

15. Document Control

Field	Value
Document Title	ai2agi Inter-Component API Specification
Version	2.0
Date	May 25, 2026
Author	Alex Osterneck, CLA, MSCS, MSIT
Organization	Ai70000, Ltd.
Classification	CONFIDENTIAL — PROPRIETARY INTELLECTUAL PROPERTY — Ai70000, Ltd.
Genesis Document	How we go from AI to AGI (ai2agi), March 2, 2025 (ai70000-030225)
Supersedes	v1.0 drafts: 1_e_generalization_API, 2_e_reasoning_API, 4_dtb_API, 5_cv_adapt_API — all May 2026
Next Revision	Version 2.1 — upon completion of G_enhanced (C1) source code and DTB (C4) biophysical ODE pipeline

CONFIDENTIAL — PROPRIETARY IP — Ai70000, Ltd. — Version 2.0 — May 24, 2026